Trends

"Embedded AI is No Longer Waiting for the Cloud"

Transformer-level AI running natively on microcontrollers was once considered impossible. Today, it is becoming a reality, with implications that extend far beyond early expectations. In this interview, ***Henrik Flodell of Alif Semiconductor*** tells ***EFY's Akanksha Sondhi Gaur*** how the new ExecuTorch–PyTorch pipeline enables developers to deploy real-time, high-accuracy models on tiny embedded devices without architectural compromises.

Henrik Flodell
Senior Product Marketing Director, Alif Semiconductor

Q. What industry shifts led to this collaboration around ExecuTorch?

A. Over the last few years, there has been a surge in the number of developers seeking to deploy PyTorch-based models on embedded devices. PyTorch, being Python-driven and developer-friendly, has emerged as the preferred framework for AI research and rapid prototyping.

Until recently, however, it lacked a clean path for deploying models directly on constrained microcontrollers due to quantisation challenges, float-to-integer conversion limitations, and accuracy losses. ExecuTorch changes this.

Designed by the team behind PyTorch, ExecuTorch provides a runtime optimised for constrained systems, allowing cloud-trained models to be deployed on edge devices with minimal accuracy loss.

Q. How is collaboration between Meta, Arm, and Alif strengthening the embedded AI ecosystem?

A. Together, Meta, Arm, and Alif are enabling a direct path for deploying PyTorch models onto microcontrollers, supported by a unified operator backend and the industry's first transformer-capable microcontrollers.

Q. Why is PyTorch emerging as the top choice for edge developers?

A. Its rise within the embedded and edge AI community is driven by two key factors: a Python-first design aligned with data science workflows, and a mature tooling ecosystem that accelerates development.

ExecuTorch acts as the missing link, bridging the gap between cloud-scale PyTorch workflows and ultra-constrained microcontrollers. It allows direct deployment without redesigning model graphs or rewriting large portions of code, simplifying workflows.

Q. Why ExecuTorch and not other conversion tools? What sets it apart?

A. Other methods for deploying PyTorch models on embedded hardware have existed, typically relying on conversion into formats supported by alternative ML runtimes. However, these approaches consistently struggled to preserve full model accuracy after conversion.

ExecuTorch overcomes this limitation by enabling high-fidelity conversion with minimal accuracy loss, while directly mapping PyTorch operators without requiring architectural compromises. It includes native support for quantised deployment for low-power MCUs. As an open source project maintained by